MODULE 18 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

Scope and the global keyword

Where a name exists, why assigning inside a function makes a new local, the UnboundLocalError trap, and what global changes.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 18 of 28

admin@sthithapragna.com

If a function **assigns** to a name anywhere in its body, that name is local for the **whole** function. Reading an outside name is fine. Assigning to one needs global.

A function can read a global variable

```
1  x = 1
2  def show():
3      print(x + 4)          # x is only read, so the global x is used
4  show()                    # 5
5  print(x)                  # 1
```

1 A name assigned outside every function is global to the file.

3 show never assigns x, so it looks x up in the global scope.

Assignment creates a new local that hides the global

```
1  num = 1
2  def f():
3      num = 3                # a new local num
4      print(num, end=" ")
5  f()
6  print(num)                 # output: 3 1
```

3 Assigning inside f creates a local num. The global num is not touched.

6 Outside, the global num is still 1.

Locals and parameters exist only during the call

```
1  def f(a):
2      y = 10
3  f(1)
4  print(y)                    # NameError: name 'y' is not defined
```

1-2 Both the parameter a and the variable y are local to f.

4 Once f returns, its local names are gone. print(a) would fail the same way.

Reading a name before its local assignment

```
1  num = 1
2  def f():
3      num = num + 3          # UnboundLocalError
4  f()
```

3 num is assigned in f, so num is local for all of f. The right-hand side reads the local, which has no value yet.

Python 3.11 onwards: cannot access local variable 'num' where it is not associated with a value. Older: local variable 'num' referenced before assignment.

The assignment does not have to come first

READ ONLY

Uses the global

```
1  x = 5
2  def f():
3      print(x)
4  f()
```

5. f never assigns x, so x is global.

READ, THEN ASSIGN

Fails at the print

```
1  x = 5
2  def f():
3      print(x)
4      x = 2
5  f()
```

UnboundLocalError, raised at print(x). The later x = 2 makes x local everywhere in f.

THE TIEBREAKER

Python decides local or global by scanning the whole body when def runs, not line by line.

global lets a function assign to the outside name

```
1  count = 0
2  def tick():
3      global count            # count here means the global count
4      count += 1
5  tick()
6  tick()
7  print(count)               # 2
```

3 Declares that count in this function refers to the global name. No local is created.

4 Now reading and assigning both use the global.

A global declared in a function can be created there

```
1  def setup():
2      global mode
3      mode = "fast"           # creates the global mode
4  setup()
5  print(mode)               # fast
```

2-3 mode did not exist before. The assignment creates it in the global scope.

5 print(mode) before setup() runs would be a NameError.

Changing a global list needs no global keyword

CHANGE THE OBJECT

Works without global

```
1  data = [1]
2  def add():
3      data.append(2)
4  add()
5  print(data)
```

[1, 2]. data is only read, to find the list; then the list is changed.

ASSIGN THE NAME

Creates a local

```
1  data = [1]
2  def reset():
3      data = []
4  reset()
5  print(data)
```

[1]. data = [] makes a new local. The global list is untouched.

THE TIEBREAKER

global is about assigning to a name. Calling a method, or setting data[0], is not assigning to data.

The test: is the name itself assigned?

INSIDE THE FUNCTION	NEEDS GLOBAL?	WHY
`print(data)`	no	only reads the name
`data.append(2)`	no	reads the name, changes the object
`data[0] = 9`	no	assigns to an element, not to the name
`data = [9]`	yes	assigns to the name: otherwise a local
`data += [2]`	yes	+= is an assignment: UnboundLocalError

THE TRAP

The last row surprises people: for a list, += changes it in place, but it still counts as assigning the name.

The lookup order: local, then global, then built-in

SCOPE	WHAT LIVES THERE	EXAMPLE
Local	names assigned in the current function, and its parameters	y = 10 inside f
Enclosing	names in an outer function, for nested functions	beyond the syllabus
Global	names assigned at the top level of the file	count = 0
Built-in	names Python provides	len, print, range

ON THE EXAM

The first scope that has the name wins. That is why a local or global named len hides the built-in len.

if and for blocks do not create a scope

```
1  if True:
2      inside = 1
3  for i in range(3):
4      pass
5  print(inside, i)          # 1 2: both are global names
```

1-4 Names assigned in if, while and for blocks belong to the surrounding scope.

5 At the top level of a file, that scope is global.

What goes wrong

CODE	ERROR
`reading a local, like y, after the function has returned`	NameError
`x = x + 1 inside a function, with no global`	UnboundLocalError
`print(x) then global x, in the same function`	SyntaxError: used prior to global declaration
`def f(x): global x`	SyntaxError: name 'x' is parameter and global

ON THE EXAM

UnboundLocalError is a kind of NameError. Module 20 places both in the exception hierarchy.

What to carry into the exam

- Assigned anywhere in a function: **local for the whole function**.
- Only read: the global is used. Locals and parameters vanish after the call.
- Read before the local assignment: **UnboundLocalError**. data += [2] counts as assigning.
- global lets a function assign, or even create, the outside name.
- append, data[0] = … and other changes to a global object need no global.

What is the output of this program?

```
1  level = 1
2  def up():
3      level = 5
4      print(level, end=" ")
5
6  up()
7  print(level)
```

A 5 1

B 5 5

C 1 1

D The code raises an UnboundLocalError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  level = 1
2  def up():
3      level = 5
4      print(level, end=" ")
5
6  up()
7  print(level)
```

level = 5 creates a local inside up. The global level stays 1.

A. 5 1

The local prints 5; the global, untouched, prints 1.

B. 5 5

Treats the assignment as changing the global. That needs global.

C. 1 1

Inside up, the local level hides the global.

D. The code raises an UnboundLocalError

level is assigned before it is read, so the local has a value.

THE SIGNAL

Assigning inside a function without global makes a separate local variable.

What happens when this program runs?

```
1  total = 10
2  def add(n):
3      total = total + n
4      return total
5
6  print(add(5))
```

A 15 is printed

B The code raises an UnboundLocalError

C 10 is printed

D 5 is printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  total = 10
2  def add(n):
3      total = total + n
4      return total
5
6  print(add(5))
```

total is assigned in add, so it is local throughout add. The right side reads it before it has a value.

A. 15 is printed
What the author meant. It would need global total.

B. The code raises an UnboundLocalError
The local total is read on the right before it is assigned.

C. 10 is printed
The function never gets as far as returning.

D. 5 is printed
Treats the local total as starting at 0. It has no value at all.

THE SIGNAL

x = x + something inside a function, with no global, is always an UnboundLocalError.

What is the output of this program?

```
1  score = 0
2  def win():
3      global score
4      score += 3
5  def lose():
6      score = -1
7
8  win()
9  lose()
10 win()
11 print(score)
```

A 6

B -1

C 5

D 3

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  score = 0
2  def win():
3      global score
4      score += 3
5  def lose():
6      score = -1
7
8  win()
9  lose()
10 win()
11 print(score)
```

win changes the global twice: 0, 3, 6. lose only sets its own local.

A 6

lose assigns a local, so only the two wins count.

B -1

Treats lose as changing the global. It has no global line.

C 5

Applies lose's -1 to the global between the wins.

D 3

Counts only one win.

THE SIGNAL

global applies only in the function that declares it. Every other function follows the normal rule.

What is the output of this program?

```
1  items = []
2  def add(x):
3      items.append(x)
4      return len(items)
5
6  add("a")
7  print(add("b"), items)
```

A `1 ['b']`

B `2 ['a', 'b']`

C The code raises an UnboundLocalError

D `2 []`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  items = []
2  def add(x):
3      items.append(x)
4      return len(items)
5
6  add("a")
7  print(add("b"), items)
```

items is never assigned inside add, so it is the global list, and append changes it.

A `1 ['b']`
Assumes each call starts with a fresh list. The global list persists.

B `2 ['a', 'b']`
append changes the one global list; no assignment, so no global needed.

C The code raises an UnboundLocalError
There is no assignment to items, so it is not local.

D `2 []`
The changes are made to the global list itself.

THE SIGNAL

Calling a method on a global object is reading the name, not assigning it.

What happens when this program runs?

```
1  data = [1]
2  def grow():
3      data += [2]
4
5  grow()
6  print(data)
```

A [1, 2] is printed

B [1] is printed

C The code raises an UnboundLocalError

D [2] is printed

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  data = [1]
2  def grow():
3      data += [2]
4
5  grow()
6  print(data)
```

+= is an assignment to data, so data is local in grow, and it is read before it has a value.

A [1, 2] is printed
Would be right with global data, or with data.append(2).

B [1] is printed
The function does not quietly fail. It raises.

C The code raises an UnboundLocalError
+= assigns the name, making it local; reading it first fails.

D [2] is printed
Treats the local data as an empty list. It has no value.

THE SIGNAL

For scope, += is an assignment, even when it would change a list in place.

> Consider scope in Python functions in general.

Which two statements are true? (Choose two.)

A. A name assigned inside a function is local unless declared global

B. A function cannot read a global variable without the global keyword

C. A function's parameters can still be used after it returns

D. Changing an element of a global list inside a function needs no global keyword

◆ PAUSE AND DECIDE

Which two statements are true? (Choose two.)

A A name assigned inside a function is local unless declared global

The rule of this module.

B A function cannot read a global variable without the global keyword

Reading needs no keyword; only assigning does.

C A function's parameters can still be used after it returns

Parameters are local, and vanish with the call.

D Changing an element of a global list inside a function needs no global keyword

data[0] = x assigns an element, not the name.

THE SIGNAL

global is needed only to assign the name itself. Reading, and changing the object, never need it.